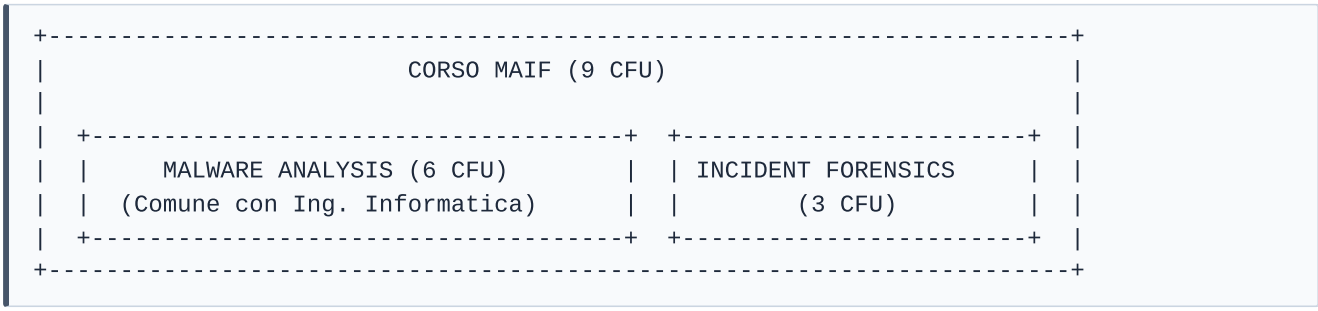
MALWARE ANALYSIS & INCIDENT FORENSICS

Appunti Integrati di Lezione – Lezione 1: Introduzione e Architettura x86

PARTE 1: INTRODUZIONE AL CORSO E PANORAMA DEL MALWARE

1. Informazioni Generali e Struttura del Corso

- **Articolazione dei Crediti (CFU)**:
  - **Malware Analysis (MA)**: 6 CFU (obbligatorio per Cyber Security e Ingegneria Informatica).
  - **Incident Forensics**: 3 CFU aggiuntivi (specifico per il corso di laurea in Cyber Security).
- **Docenti del Corso**: Prof. Daniele Cono D'Elia (6 CFU) e Prof. Leonardo Querzoni (3 CFU).
- **Organizzazione Didattica e Orari**:
  - **Martedì (14:00 - 16:00)**: Lezione teorica (MA & MAIF) – Aula 101, Palazzina D, Viale Regina Elena 295.
  - **Giovedì (14:00 - 17:00)**: Lezione/Seminari (Solo MAIF) – Aula 201, Palazzina D, Viale Regina Elena 295.
  - **Venerdì (14:00 - 17:00)**: Laboratorio Pratico (MA & MAIF) – Lab 16, Via Tiburtina 205.
- **Testo di Riferimento**: *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software* (M. Sikorski, A. Honig - No Starch Press). I principi metodologici di analisi rimangono invariati rispetto alle basi del testo.



2. Ambiente di Lavoro (VM) e Modalità d'Esame

Macchina Virtuale di Laboratorio

- **Configurazione**: Sistema operativo **Windows 10** (build datata, senza aggiornamenti automatici e con Windows Defender disattivato) per evitare interferenze durante la detonazione del malware.
- **Hypervisor consigliato**: **VirtualBox** (è vivamente raccomandato creare subito uno *snapshot* iniziale pulito per ripristinare il sistema a seguito dell'esecuzione dei sample).
- **Dispositivi Apple Silicon (M1/M2/M3)**: Richiedono emulazione; per i Mac recenti (M3+) la fluidità è accettabile tramite le procedure d'installazione dedicate fornite su Google Classroom.

Modalità di Valutazione dell'Esame

1. **Prova Pratica (3 ore)**: Analisi dinamica/statica su un sample di malware sintetico personalizzato. Risposta a domande analitiche e stesura di un report. Voto espresso in trentesimi (≥ 18). Valido per tutti gli studenti (6 CFU e 9 CFU).
2. **Questionario VERO/FALSO (Solo per 9 CFU - MAIF)**: Test di verifica concettuale a valutazione binaria (Pass/Fail). Non influisce sulla media del voto numerico, ma è vincolante per la verbalizzazione dei 9 CFU.

3. Case Studies Storici ed Evoluzione delle Minacce

L'evoluzione del malware negli ultimi decenni ha mostrato un drastico cambio di paradigma: da attacchi dimostrativi ad un modello di business altamente redditizio e strutturato.

```
Evoluzione del Malware:
[Data Breaches Massive] ---> [Leak di Armi Cyber (0-day)] ---> [Ransomware-as-a-Service (RaaS)]
    (es. Yahoo, Equifax)          (es. Shadow Brokers)              (es. LockBit 3.0)
```

1. **Yahoo Data Breach (2016)**:
   - Sottrazione di credenziali di oltre **500 milioni di utenti**.
   - Originato da conflitti interni tra il team di sicurezza ("The Paranoids") e i dirigenti aziendali, che hanno preferito l'usabilità rispetto ai costi della sicurezza.
2. **Equifax Data Breach (settembre 2017)**:
   - Esposizione dei dati finanziari di **>160 milioni di cittadini americani**.
   - Causa: mancata applicazione di una patch di sicurezza su una vulnerabilità nota in un server web (problema ignorato per oltre 2 anni).
3. **The Shadow Brokers & WannaCry (2016 - Maggio 2017)**:
   - Il gruppo *Shadow Brokers* mette all'asta (e poi diffonde gratuitamente il 14/04/2017) un archivio di exploit avanzati (tra cui **EternalBlue**) sottratti all'Equation Group (legato alla NSA).
   - **WannaCry (Maggio 2017)**: Ransomware che ha sfruttato EternalBlue (protocollo SMB) per infettare indiscriminatamente centinaia di migliaia di computer (inclusi i sistemi sanitari del NHS britannico).
   - *Debolezza strutturale*: WannaCry conteneva nel codice un interruttore di blocco (*kill-switch*), basato sulla verifica della registrazione di un dominio specifico non registrato.
4. **LockBit & il Ransomware-as-a-Service (RaaS)**:
   - Modello "Ransomware-as-a-Service": la piattaforma viene affittata agli affiliati in cambio di una percentuale sui riscatti.
   - **Estorsione Multipla (Triple Extortion)**:
     1. Cifratura locale dei dati.
     2. Minaccia di pubblicazione/leak di informazioni riservate (*Doxxing*).
     3. Pressione diretta sui clienti o terze parti della vittima.
   - Modello aziendale moderno: presenza di programmi di *Bug Bounty* ufficiali per chi individua falle nel codice del ransomware e supporto clienti dedicato.

4. Anatomy of an Attack: The Cyber Kill Chain

Modello teorico sviluppato da **Lockheed Martin** per strutturare le fasi sequenziali di un cyber-attacco:

```
[Reconnaissance] ---> [Weaponization] ---> [Delivery] ---> [Exploitation]
                                                                  |
 [Lateral Spread] <--- [Data Exfiltration] <--- [Callback] <--- [Persistence]
```

1. **Reconnaissance (Ricognizione)**: Identificazione delle vulnerabilità tecniche o umane della vittima.
2. **Weaponization (Armamentario)**: Creazione dell'exploit accoppiato al payload maligno.
3. **Delivery (Consegna)**: Invio dell'arma (es. e-mail di phishing, drive USB infetto).
4. **Exploitation (Sfruttamento)**: Esecuzione del codice malevolo tramite vulnerabilità o ingegneria sociale.
5. **Persistence (Persistenza)**: Tecniche usate dal malware per sopravvivere ai riavvii di sistema (es. modifica delle chiavi di Registro Windows, Task Scheduler, Cartella Esecuzione Automatica).
6. **Callback (Beaconing & C2)**: Il malware contatta il server di *Command & Control* (C2) remoto per confermare l'infezione e ricevere istruzioni.
7. **Data Exfiltration (Esfiltrazione Dati)**: Sottrazione e invio all'esterno delle informazioni sensibili.
8. **Lateral Spread (Spostamento Laterale)**: Diffusione dell'infezione verso altre macchine presenti all'interno della medesima rete locale.

Comportamenti Tipici del Malware

- **Unicità ("Single-instance")**: Il malware verifica di non essere già in esecuzione sulla macchina (usando *Mutex* o flag di sistema) per evitare conflitti o spreco di risorse.
- **Targeting & Fingerprinting**: Ispezione delle caratteristiche del sistema per verificare se la vittima risponde a criteri specifici (layout della tastiera, lingua del sistema operativo, ID del processore, presenza di particolari dispositivi). Se i criteri non corrispondono, il malware termina l'esecuzione o mostra un comportamento benigno.
- **Obfuscation & Evasion**: Mascheramento del codice (cifratura, packing, code injection) per ostacolare l'analisi e i sistemi antivirus/EDR.

PARTE 2: LINGUAGGIO ASSEMBLY X86 & ARCHITETTURA DELLA CPU

1. Architettura della CPU e Concetti Base

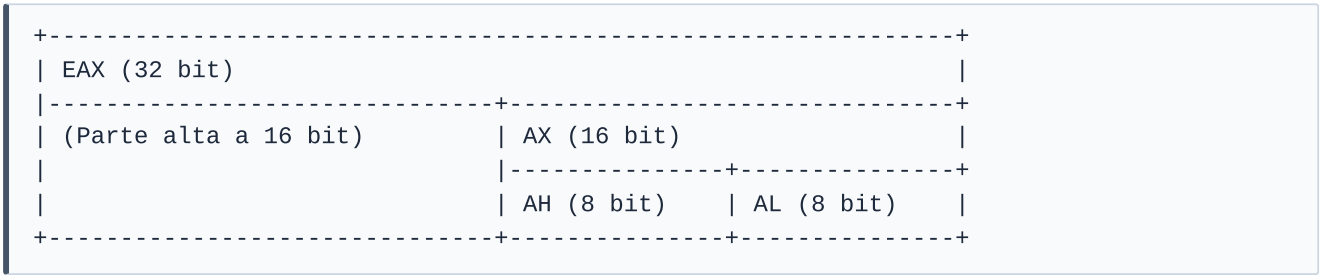
- **Instruction Set Architecture (ISA)**: Interfaccia astratta tra l'hardware e il software. Definisce i registri, le istruzioni supportate e le modalità di indirizzamento della memoria.
- **Dimensione della Parola (Word Size)**:
  - La dimensione nativa per le architetture x86 (IA-32) è **32 bit (Dword)**.
  - Tipi di dato fondamentali: **Byte** (8 bit), **Word** (16 bit), **Dword** (32 bit), **Qword** (64 bit).
- **Endianness (Ordinamento dei Byte in Memoria)**:
  - L'architettura x86 utilizza la codifica **Little-Endian**: il byte *meno significativo* viene memorizzato all'indirizzo di memoria più basso.
  - *Nota*: I registri del debugger mostrano i dati nell'ordine umano (Big-Endian per la lettura), ma la rappresentazione esadecimale in memoria appare invertita a coppie di byte.

```
Esempio: Memorizzazione del valore 32-bit 0xABADCAFE all'indirizzo 0x1000

     Big-Endian (Umano / Reti):     [0x1000]: AB  [0x1001]: AD  [0x1002]: CA  [0x1003]:
FE
     Little-Endian (x86):           [0x1000]: FE  [0x1001]: CA  [0x1002]: AD  [0x1003]:
AB
```

2. I Registri General-Purpose (GPR) in x86

L'architettura x86 dispone di **8 registri generali a 32 bit**, la cui struttura consente l'accesso alle porzioni a 16 e 8 bit per compatibilità:



Registro	Ruolo Principale	Accessibilità Sub-porzioni
`EAX`	Accumulatore / Valore di Ritorno	AX (16-bit), AH (8-bit), AL (8-bit)
`EBX`	Base Register / Dati generali	BX (16-bit), BH (8-bit), BL (8-bit)
`ECX`	Contatore (Loop, Stringhe, Rep)	CX (16-bit), CH (8-bit), CL (8-bit)
`EDX`	I/O, Estensione Operazioni Aritm.	DX (16-bit), DH (8-bit), DL (8-bit)
`ESI`	Source Index (Copia/Stringhe)	SI (16-bit)
`EDI`	Destination Index (Copia/Stringhe)	DI (16-bit)
`ESP`	Stack Pointer (Punta alla Cima)	SP (16-bit)
`EBP`	Base Pointer (Frame di Funzione)	BP (16-bit)

- **Program Counter (`EIP`)**: Contiene l'indirizzo della prossima istruzione da eseguire (non modificabile direttamente tramite istruzioni `mov`).
- **Status Register (`EFLAGS`)**: Registra lo stato logico della CPU a seguito di operazioni aritmetico-logiche:
  - **ZF (Zero Flag)**: Impostato a `1` se il risultato dell'operazione è zero.
  - **SF (Sign Flag)**: Impostato al bit più significativo (indica un risultato negativo).
  - **CF (Carry Flag)**: Indica trabocco per operazioni non segnate (unsigned).
  - **OF (Overflow Flag)**: Indica trabocco per operazioni con segno (signed).

3. Mappa di Memoria di un Processo a 32-bit (Windows)

Nei sistemi a 32 bit, lo spazio d'indirizzamento virtuale totale è pari a 2^{32} = 4 GB. Di default, l'ambiente di memoria è suddiviso nel seguente modo:

```
0x00000000 +-----------------------------------+
           | Program Image (.text, .data, ecc.)|
           +-----------------------------------+
           | Heap (Allocazione dinamica)       |
           |                |                  |
           |                v                  |
           |                                   |
           |                ^                  |
           |                |                  |
           | Stack (Cresce verso il basso)     |
           +-----------------------------------+
           | DLL Caricate / TEB & PEB          |
0x7FFFFFFF +-----------------------------------+ (Limite User Space: 2 GB)
0x80000000 |                                   |
           | KERNEL SPACE                      |
           | (Inaccessibile da User Space)     |
0xFFFFFFFF +-----------------------------------+ (Kernel Space: 2 GB)
```

> **NOTA FONDAMENTALE SULLO STACK:** Lo Stack cresce fisicamente **dall'alto verso il basso** (verso indirizzi di memoria decrescenti). L'inserimento di un dato (`push`) provoca un **decremento** dell'indicatore `ESP` .

4. Set di Istruzioni x86 Fondamentali

A. Trasferimento Dati & Aritmetica

- `mov dest, src` : Copia il contenuto di `src` in `dest` .
- `push src` : Decrementa `ESP` di 4 byte e memorizza `src` in `[ESP]` .
- `pop dest` : Copia il valore in `[ESP]` dentro `dest` e incrementa `ESP` di 4 byte.
- `add dest, src` / `sub dest, src` : Somma/Sottrazione (dest = dest ± src).
- `inc dest` / `dec dest` : Incremento/Decremento unitario.
- `lea dest, [expr]` (Load Effective Address): Calcola l'indirizzo di memoria dell'espressione e salva l'indirizzo risultante (non il valore contenuto!) in `dest` . Usato frequentemente per il calcolo rapido di espressioni aritmetiche:

  ```
  lea ecx, [eax + 2*ebx - 1]   ; Calcola il risultato matematico in 1 solo ciclo di cl
  ock
  ```

B. Logica & Manipolazione Bit

- `and` / `or` / `xor` : Operazioni bitwise standard.
- `xor eax, eax` : Idioma assembly utilizzato per azzerare rapidamente un registro (più efficiente e compatto rispetto a `mov eax, 0`).

C. Controllo del Flusso

- **Unconditional Branching**:
  - `jmp target`: Trasferisce il controllo direttamente all'indirizzo specificato.
- **Conditional Branching** (Eseguito a seguito di una `cmp dest, src` o `test dest, src`):
  - `cmp dest, src`: Esegue `dest - src` modificando i flag in `EFLAGS` senza alterare i registri.
  - `test dest, src`: Esegue un `AND` bitwise modificando i flag (usato comunemente come `test eax, eax` per verificare se un valore è nullo).
  - `je / jz`: Salta se uguale (Zero Flag = 1).
  - `jne / jnz`: Salta se non uguale (Zero Flag = 0).
  - `jg / jle`: Salto condizionato con segno (Greater / Less or Equal).
  - `ja / jbe`: Salto condizionato senza segno (Above / Below or Equal).

5. Convenzioni di Chiamata (Calling Conventions)

Le convenzioni di chiamata stabiliscono la modalità di passaggio dei parametri alle funzioni, la gestione del valore di ritorno e la responsabilità del ripristino dello Stack Pointer (`ESP`).

```
    Stack Frame durante la chiamata a una funzione:

    | ...                       |
    | Parametro 2               |  [ebp + 12]
    | Parametro 1               |  [ebp + 8]
    | Indirizzo di Ritorno      |  [ebp + 4]   <-- Salvato automaticamente da CALL
    | EBP Precedente (Saved)    |  [ebp]       <-- Puntato da EBP corrente
    | Variabile Locale 1        |  [ebp - 4]
    | Variabile Locale 2        |  [ebp - 8]   <-- Puntato da ESP
```

Confronto delle Convenzioni Principali:

Proprietà	`cdecl` (C Standard)	`stdcall` (Win32 API)	`fastcall`
Passaggio Argomenti	Stack (da destra a sinistra)	Stack (da destra a sinistra)	Registri `ECX`, `EDX` (i primi 2), poi Stack
Pulizia dello Stack	**Chiamante (Caller)** via `add esp, N`	**Chiamato (Callee)** via `ret N`	**Chiamato (Callee)** via `ret N`
Registri Preservati (Callee-Save)	`EBX`, `ESI`, `EDI`, `EBP`	`EBX`, `ESI`, `EDI`, `EBP`	`EBX`, `ESI`, `EDI`, `EBP`
Valore di Ritorno	`EAX` (32-bit)	`EAX` (32-bit)	`EAX` (32-bit)

6. Tecniche Anti-Disassembly (Cenni Avanzati)

I creatori di malware impiegano costrutti per ingannare i disassemblatori statici (come le versioni base di IDA Pro o Ghidra):

1. **Schema `call $+5`**:

```
call $+5          ; Spinge sullo stack l'indirizzo dell'istruzione successiva (pop e
ax)
pop eax           ; EAX contiene ora l'indirizzo dinamico di esecuzione corrente
```

2. **Jumps sfalsati ed esecuzione sovrapposta**: Inserimento di byte civetta (*junk bytes*) ed esecuzione di istruzioni di salto (`jmp`) che puntano a metà di un'altra istruzione formale. Questo disallinea il disassemblatore lineare che interpreta erroneamente la sequenza di opcodes successiva.

SINTESI PER IL RIPASSO (PUNTI CHIAVE DELL'ESAME):

- **x86 = Little-Endian**: In memoria i byte di una Dword sono invertiti.
- **Stack**: Cresce verso indirizzi bassi (`push` sottrae 4, `pop` somma 4 a `ESP`).
- **`cdecl` vs `stdcall`** : In `cdecl` lo stack è sistemato dal chiamante (`add esp, X`); in `stdcall` dal chiamato (`ret X`).
- **`LEA`** : Calcola la matematica dell'indirizzo senza accedere alla RAM.
- **Persistenza**: Operazione fondamentale del malware per sopravvivere ai riavvii di sistema.
- **Threat Intelligence & IOC**: Oggetto del modulo da 3 CFU per la caratterizzazione delle minacce.